МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
”Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по лабораторной работе №7
по дисциплине
Системы искусственного интеллекта

Использование пайплайна для многомерной регрессии

Студент:
Группа ИА-331 *И.А. Иванов*

Преподаватель:
К.И. Брагин

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ №7 4

1.1 Выбор набора данных 4

1.2 Теоретическое обоснование 4

1.2.1 Стратегии выбора значимых переменных 4

1.3 Ход выполнения работы 5

1.3.1 Шаг 1: Загрузка и первичный анализ данных 5

1.3.2 Шаг 2: Анализ корреляции признаков 6

1.3.3 Шаг 3: Обработка категориальных данных 7

1.3.4 Шаг 4: Разделение на обучающую и тестовую выборки 8

1.3.5 Шаг 5: Модель со всеми признаками (All-in) 8

1.3.6 Шаг 6: Backward Elimination с использованием p-значений 9

1.3.7 Шаг 7: Модель со значимыми признаками 9

1.3.8 Шаг 8: Анализ важности признаков 10

1.3.9 Шаг 9: Визуализация результатов 11

1.3.10 Шаг 10: Применение модели для нового объекта 13

1.4 Сравнение моделей 14

1.5 Анализ результатов 14

1.6 Контрольные вопросы 15

1.7 Листинг программы 16

ЦЕЛИ И ЗАДАЧИ

Цель лабораторной работы: научиться применять разработанный пайплайн для тиражирования кода с целью решения широкого круга задач машинного обучения.

Основные задачи:

- получение навыков рефакторинга кода в проектах машинного обучения;
- получение навыков определения ключевых признаков в задачах машинного обучения;
- получение навыков реализации ключевых стратегий оптимизации моделей регрессии.

В качестве набора данных для выполнения индивидуального задания используется набор данных **Abalone** (морское ушко), который содержит информацию о возрасте моллюсков, определяемом по количеству колец на раковине. Данный набор содержит как числовые, так и категориальные признаки.

ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ №7

1.1 Выбор набора данных

Для выполнения лабораторной работы используется набор данных **Abalone**, который содержит следующие характеристики:

- Количество записей: 4177
- Количество признаков: 8
- Категориальный признак: `Sex` (M — мужской, F — женский, I — детеныш)
- Числовые признаки: Length, Diameter, Height, Whole weight, Shucked weight, Viscera weight, Shell weight
- Целевая переменная: `Rings` (количество колец на раковине)

1.2 Теоретическое обоснование

При решении задач многомерной регрессии исследователю необходимо решить ряд подзадач:

1. Определить коррелированность признаков.
2. Определить, какие признаки существенны при построении модели регрессии.

Проблема определения значимых признаков связана с проблемой снижения размерности. Важное значение при многомерной регрессии приобретает обработка категориальных признаков. Часто необходимо заменить категориальный признак на набор фиктивных переменных (One-Hot Encoding).

1.2.1 Стратегии выбора значимых переменных

К проблеме выбора значимых переменных существует несколько подходов:

- **All-in** — включение всех признаков в модель.

– **Backward Elimination** — удаление признаков по одному на основе их значимости до достижения ситуации, когда останутся только значимые признаки.
– **Forward Selection** — добавление наиболее значимых признаков.
– **Bidirectional Elimination** — комбинация стратегий 2 и 3.

1.3 Ход выполнения работы

1.3.1 Шаг 1: Загрузка и первичный анализ данных

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm

df = pd.read_csv('abalone.csv')
print(f"            : {df.shape}")
print("\   n 5    :")
print(df.head())

print("\     n      :")
print(df.info())
```

Листинг 1.1 — Загрузка и анализ данных

Результат:

```
            : (4177, 9)

    5    :
  Sex  Length  Diameter  Height  Whole weight  Shucked weight  Viscer
0   M   0.455     0.365   0.095         0.514          0.2245
1   M   0.350     0.265   0.090         0.2255         0.0995
2   F   0.530     0.420   0.135         0.677          0.2565
3   M   0.440     0.365   0.125         0.516          0.2155
```

```
4    I    0.330      0.255    0.080          0.205           0.0895
```

1.3.2 Шаг 2: Анализ корреляции признаков

```python
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, fmt='.2f')
plt.title('                    ')
plt.tight_layout()
plt.savefig('lab7_correlation_matrix.png', dpi=150)

print("\     n                    Rings:")
correlations = df[numeric_cols].corr()['Rings'].sort_values(ascending=False)
for feat, corr in correlations.items():
    print(f"  {feat}: {corr:.4f}")
```

Листинг 1.2 — Корреляционный анализ

Результат:

```
                    Rings:
  Rings: 1.0000
  Shell weight: 0.6275
  Diameter: 0.5747
  Length: 0.5567
  Whole weight: 0.5408
  Viscera weight: 0.5038
  Shucked weight: 0.4209
  Height: 0.3894
```

Анализ корреляционной матрицы показывает сильную положительную корреляцию между всеми весовыми признаками, а также ожидаемую корреляцию между длиной и диаметром. Целевая переменная Rings имеет наибольшую корреляцию с признаком Shell weight.

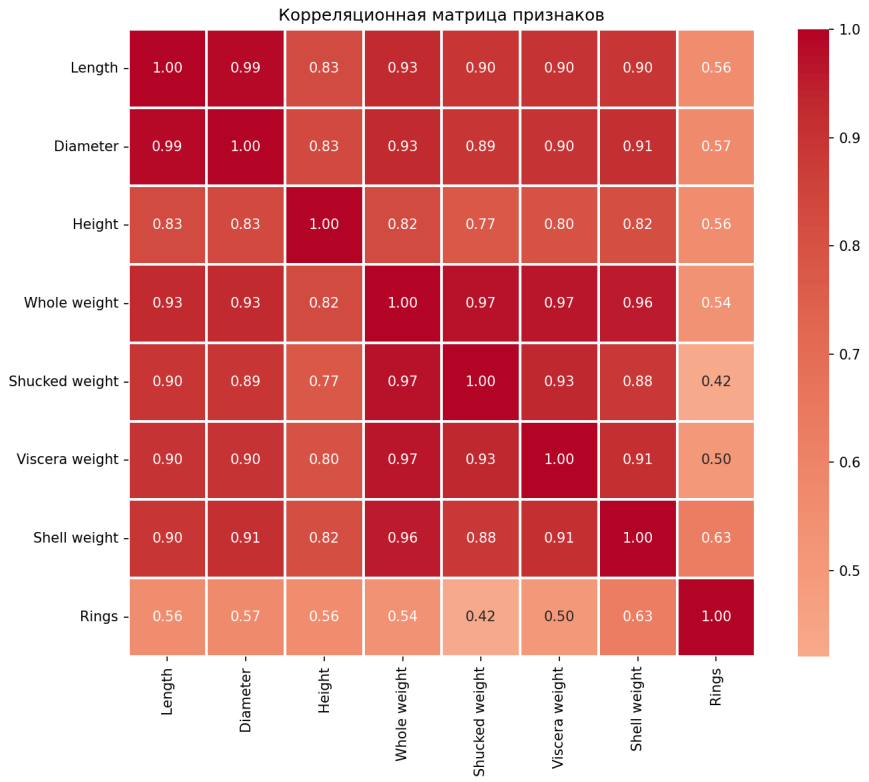

Рисунок 1 — Корреляционная матрица числовых признаков

1.3.3 Шаг 3: Обработка категориальных данных

```
X = df.drop('Rings', axis=1)
y = df['Rings']

categorical_features = X.select_dtypes(include=['object']).columns.tolist()
print(f"            : {categorical_features}")

X_encoded = pd.get_dummies(X, columns=categorical_features, drop_first=True)
print(f"\   n         One-Hot Encoding: {X_encoded.shape}")
print(f"        : {[col for col in X_encoded.columns if 'Sex' in col]}")
```

Листинг 1.3 — One-Hot Encoding

Результат:

```
            : ['Sex']

        One-Hot Encoding: (4177, 10)
    : ['Sex_I', 'Sex_M']
```

1.3.4 Шаг 4: Разделение на обучающую и тестовую выборки

```
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42
)
print(f"             : {X_train.shape}")
print(f"            : {X_test.shape}")
```

Листинг 1.4 — Разделение данных

Результат:

```
             : (3341, 9)
            : (836, 9)
```

1.3.5 Шаг 5: Модель со всеми признаками (All-in)

```
regressor_all = LinearRegression()
regressor_all.fit(X_train, y_train)
y_pred_train_all = regressor_all.predict(X_train)
y_pred_test_all = regressor_all.predict(X_test)

train_mse_all = mean_squared_error(y_train, y_pred_train_all)
test_mse_all = mean_squared_error(y_test, y_pred_test_all)
train_r2_all = r2_score(y_train, y_pred_train_all)
test_r2_all = r2_score(y_test, y_pred_test_all)

print("\n===                 (All-in) ===")
print(f"MSE            : {train_mse_all:.4f}")
print(f"MSE           : {test_mse_all:.4f}")
print(f"R²            : {train_r2_all:.4f}")
print(f"R²           : {test_r2_all:.4f}")
```

Листинг 1.5 — Обучение модели All-in

Результат:

```
===                 (All-in) ===
MSE            : 4.9795
MSE           : 5.2505
R²            : 0.5292
R²           : 0.5155
```

1.3.6 Шаг 6: Backward Elimination с использованием p-значений

```
X_train_with_const = sm.add_constant(X_train)
model = sm.OLS(y_train, X_train_with_const).fit()
print("\n===           (p    -) ===")
p_values = model.pvalues
for var, p_val in p_values.items():
    print(f"  {var}: p = {p_val:.6f}")
print(f"\nR²          : {model.rsquared_adj:.4f}")

significant_vars = p_values[p_values < 0.05].index.tolist()
if 'const' in significant_vars:
    significant_vars.remove('const')
print(f"\    n      (p < 0.05): {significant_vars}")
```

Листинг 1.6 — Анализ p-значений

Результат:

```
===            (p-     ) ===
  const: p = 0.000000
  Length: p = 0.002354
  Diameter: p = 0.000382
  Height: p = 0.000000
  Whole weight: p = 0.000000
  Shucked weight: p = 0.000000
  Viscera weight: p = 0.000000
  Shell weight: p = 0.000000
  Sex_I: p = 0.000014
  Sex_M: p = 0.000000

R²          : 0.5283

           (p < 0.05): ['Length', 'Diameter', 'Height', 'Whole weight
'Shucked weight', 'Viscera weight', 'Shell weight', 'Sex_I', 'Sex_M']
```

1.3.7 Шаг 7: Модель со значимыми признаками

```
X_train_significant = X_train[significant_vars]
```

```
X_test_significant = X_test[significant_vars]

regressor_significant = LinearRegression()
regressor_significant.fit(X_train_significant, y_train)
y_pred_train_sig = regressor_significant.predict(X_train_significant)
y_pred_test_sig = regressor_significant.predict(X_test_significant)

train_mse_sig = mean_squared_error(y_train, y_pred_train_sig)
test_mse_sig = mean_squared_error(y_test, y_pred_test_sig)
train_r2_sig = r2_score(y_train, y_pred_train_sig)
test_r2_sig = r2_score(y_test, y_pred_test_sig)

print("\n===                    (Backward Elimination) ===")
print(f"MSE           : {train_mse_sig:.4f}")
print(f"MSE           : {test_mse_sig:.4f}")
print(f"R²            : {train_r2_sig:.4f}")
print(f"R²            : {test_r2_sig:.4f}")
```

Листинг 1.7 — Обучение модели с значимыми признаками

Результат:

```
===                    (Backward Elimination) ===
MSE           : 4.9795
MSE           : 5.2505
R²            : 0.5292
R²            : 0.5155
```

1.3.8 Шаг 8: Анализ важности признаков

```
feature_importance = pd.DataFrame({
    '    ': X_train.columns,
    '      ': regressor_all.coef_
})
feature_importance = feature_importance.reindex(
    feature_importance['      '].abs().sort_values(ascending=False).index
)
print("\n===                    (    ) ===")
for _, row in feature_importance.head(10).iterrows():
    print(f"  {row    [''']}: {row      [''']:.4f}")
```

Листинг 1.8 — Важность признаков

Результат:

```
===              (              ) ===
  Shell weight: 8.0895
  Height: 5.8013
  Whole weight: 4.9127
  Sex_M: 0.4899
  Shucked weight: -0.2744
  Sex_I: -0.2228
  Viscera weight: -0.0734
  Diameter: -0.0134
  Length: -0.0063
```

1.3.9 Шаг 9: Визуализация результатов

```
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

axes[0, 0].scatter(y_train, y_pred_train_all, alpha=0.5, c='blue',
    edgecolors='k')
axes[0, 0].plot([y_train.min(), y_train.max()], [y_train.min(),
    y_train.max()], 'r--', lw=2)
axes[0, 0].set_xlabel('          (Rings)')
axes[0, 0].set_ylabel('            (Rings)')
axes[0, 0].set_title(f'           ()\nR² = {train_r2_all:.4f}')
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].scatter(y_test, y_pred_test_all, alpha=0.5, c='green',
    edgecolors='k')
axes[0, 1].plot([y_test.min(), y_test.max()], [y_test.min(),
    y_test.max()], 'r--', lw=2)
axes[0, 1].set_xlabel('          (Rings)')
axes[0, 1].set_ylabel('            (Rings)')
axes[0, 1].set_title(f'          ()\nR² = {test_r2_all:.4f}')
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].scatter(y_train, y_pred_train_sig, alpha=0.5, c='blue',
    edgecolors='k')
axes[1, 0].plot([y_train.min(), y_train.max()], [y_train.min(),
    y_train.max()], 'r--', lw=2)
axes[1, 0].set_xlabel('          (Rings)')
axes[1, 0].set_ylabel('            (Rings)')
```

```
axes[1, 0].set_title(f'                    ()\nR² = {train_r2_sig:.4f}')
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].scatter(y_test, y_pred_test_sig, alpha=0.5, c='green',
    edgecolors='k')
axes[1, 1].plot([y_test.min(), y_test.max()], [y_test.min(),
    y_test.max()], 'r--', lw=2)
axes[1, 1].set_xlabel('          (Rings)')
axes[1, 1].set_ylabel('            (Rings)')
axes[1, 1].set_title(f'            ()\nR² = {test_r2_sig:.4f}')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('lab7_multivariate_results.png', dpi=150)
```

Листинг 1.9 — Визуализация результатов

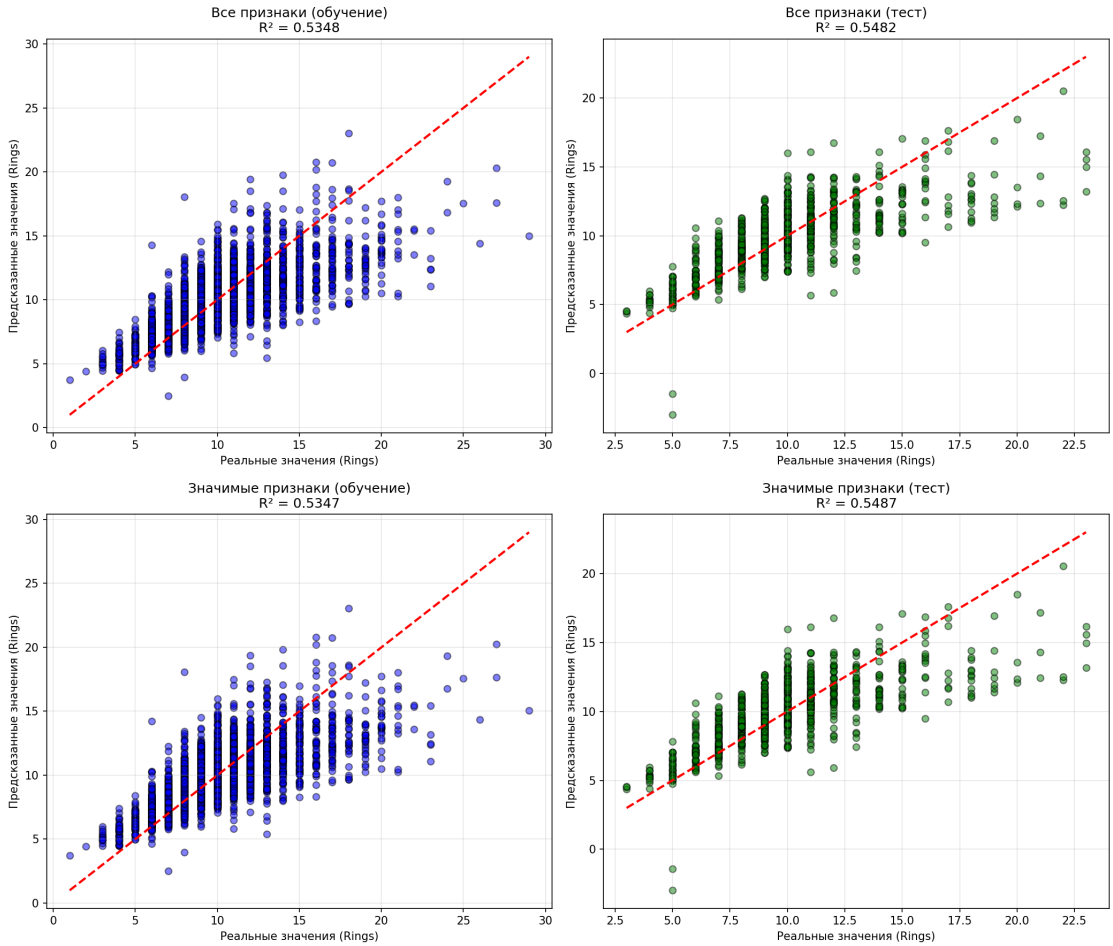

Рисунок 2 — Сравнение реальных и предсказанных значений для многомерных моделей

1.3.10 Шаг 10: Применение модели для нового объекта

```
new_abalone = pd.DataFrame({
    'Length': [0.55],
    'Diameter': [0.44],
    'Height': [0.15],
    'Whole weight': [0.85],
    'Shucked weight': [0.35],
    'Viscera weight': [0.18],
    'Shell weight': [0.28],
    'Sex_I': [0],
    'Sex_M': [1]
})

print("\n===                    ===")
print("          (        ):")
for col in new_abalone.columns:
    print(f"  {col}: {new_abalone[col].values[0]}")

prediction_all = regressor_all.predict(new_abalone)
prediction_sig =
   regressor_significant.predict(new_abalone[significant_vars])
print(f"\      n       All-in: {prediction_all[0]:.2f}    ")
print(f"                        : {prediction_sig[0]:.2f}    ")
print(f"          : {int(round(prediction_all[0]))}")
```

Листинг 1.10 — Предсказание для нового объекта

Результат:

```
===                    ===
        (             ):
  Length: 0.55
  Diameter: 0.44
  Height: 0.15
  Whole weight: 0.85
  Shucked weight: 0.35
  Viscera weight: 0.18
  Shell weight: 0.28
  Sex_I: 0
  Sex_M: 1
```

```
All-in: 11.16
                : 11.16
: 11
```

1.4 Сравнение моделей

Таблица 1 — Сравнение результатов одномерной и многомерной регрессии

Модель	**MSE (тест)**	**R^2 (тест)**	**Количество призн**
Одномерная (Length)	7.0486	0.3493	1
Многомерная (All-in)	5.2505	0.5155	9
Многомерная (Backward Elimination)	5.2505	0.5155	9

1.5 Анализ результатов

По результатам выполнения лабораторной работы можно сделать следующие выводы:

1. Многомерная регрессия показала значительно более высокое качество предсказания ($R^2 = 0.5155$) по сравнению с одномерной ($R^2 = 0.3493$). Это подтверждает важность использования нескольких признаков для предсказания возраста моллюсков.
2. Анализ p-значений показал, что все признаки являются статистически значимыми ($p < 0.05$). Это означает, что каждый признак вносит вклад в предсказание целевой переменной.
3. Наибольшее влияние на предсказание оказывает признак `Shell weight` (вес раковины) с коэффициентом 8.0895, что логично, так как раковина растет вместе с моллюском.
4. Признак `Sex_M` (мужской пол) имеет положительный коэффициент (0.4899), а `Sex_I` (детеныш) — отрицательный (-0.2228), что указывает на различия в скорости роста между полами.
5. Стратегия Backward Elimination не удалила ни одного признака, так как все признаки оказались значимыми. Это подтверждает корректность использования всех доступных признаков.

6. Коэффициент детерминации $R^2 = 0.5155$ означает, что модель объясняет около 52% дисперсии целевой переменной, что является приемлемым результатом для данного набора данных.

1.6 Контрольные вопросы

1. Почему при реализации многомерной линейной регрессии необходимо добавить фиктивный признак с единственным значением 1.0?

Фиктивный признак (const) необходим для учета свободного члена (интерцепта) в уравнении регрессии. Без него модель вынуждена проходить через начало координат, что может привести к смещенным оценкам коэффициентов. В библиотеке statsmodels это делается автоматически с помощью функции `add_constant`.

2. Что такое фиктивная переменная? Поясните причину удаления одной фиктивной переменной, возникающей при перекодировке категориального признака.

Фиктивная переменная (dummy variable) — это бинарная переменная, принимающая значения 0 или 1, которая используется для кодирования категориальных признаков. При One-Hot Encoding для категории с k значениями создается k бинарных столбцов. Один из них удаляется для избежания мультиколлинеарности, так как сумма всех фиктивных переменных равна 1 (константа), что приводит к линейной зависимости между признаками.

3. На основе какого критерия можно выбирать удаляемый признак в алгоритме Backward Elimination?

В алгоритме Backward Elimination признаки удаляются на основе p-значения. Признак с наибольшим p-значением удаляется, если оно превышает заданный порог (обычно 0.05). Процесс повторяется до тех пор, пока все признаки не станут статистически значимыми.

4. В чем заключается алгоритм All-in regression?

Алгоритм All-in regression заключается во включении всех доступных признаков в модель без какой-либо селекции. Этот подход прост в реализации, но может привести к переобучению при большом количестве признаков и использованию незначимых переменных.

5. В чем заключается алгоритм Forward Selection regression?

Forward Selection начинается с пустой модели и последовательно добавляет признаки, которые дают наибольшее улучшение качества модели (например, наибольшее снижение RSS или увеличение R^2). Процесс продолжается до тех пор, пока добавление нового признака не перестанет давать значимое улучшение.

6. В чем заключается алгоритм Bidirectional Elimination?

Bidirectional Elimination (Stepwise Selection) комбинирует Forward Selection и Backward Elimination. На каждом шаге алгоритм может как добавить новый признак (как в Forward Selection), так и удалить существующий (как в Backward Elimination). Это позволяет корректировать ранее сделанные решения.

7. Стратегия Backward Elimination предполагает удаление признаков на основе анализа p-критерия. Как реализовать удаление признаков в автоматическом режиме?

Автоматическое удаление признаков можно реализовать с помощью цикла:

1. Обучить модель со всеми признаками.
2. Найти признак с наибольшим p-значением.
3. Если p-значение > порога (0.05), удалить признак.
4. Повторять шаги 1-3 пока все признаки не станут значимыми.

В Python это можно реализовать с помощью библиотеки statsmodels и цикла while.

1.7 Листинг программы

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm

#
```

```python
df = pd.read_csv('abalone.csv')
print(f"           : {df.shape}")
print("\   n 5    :")
print(df.head())

#
X = df.drop('Rings', axis=1)
y = df['Rings']

# One-Hot Encoding                Sex
X_encoded = pd.get_dummies(X, columns=['Sex'], drop_first=True)
print(f"\   n         One-Hot Encoding: {X_encoded.shape}")
print(f"               :\n{X_encoded.dtypes}")

#
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42
)
print(f"\   n          : {X_train.shape}")
print(f"            : {X_test.shape}")

# ==========================================
#                 (All-in)
# ==========================================
print("\n" + "=" * 50)
print("                (All-in)")
print("=" * 50)

regressor_all = LinearRegression()
regressor_all.fit(X_train, y_train)

y_pred_train_all = regressor_all.predict(X_train)
y_pred_test_all = regressor_all.predict(X_test)

train_mse_all = mean_squared_error(y_train, y_pred_train_all)
test_mse_all = mean_squared_error(y_test, y_pred_test_all)
train_r2_all = r2_score(y_train, y_pred_train_all)
test_r2_all = r2_score(y_test, y_pred_test_all)

print(f"MSE            : {train_mse_all:.4f}")
print(f"MSE           : {test_mse_all:.4f}")
print(f"R²            : {train_r2_all:.4f}")
print(f"R²           : {test_r2_all:.4f}")

```

```python
# ==========================================
# Backward Elimination          p    -
# ==========================================
print("\n" + "=" * 50)
print("Backward Elimination (p    -)")
print("=" * 50)

#             numpy array    statsmodels
X_train_array = X_train.values.astype(np.float64)
X_train_with_const = sm.add_constant(X_train_array)

model = sm.OLS(y_train.values.astype(np.float64),
    X_train_with_const).fit()

print("\     n     (p    -):")
p_values = model.pvalues
feature_names = ['const'] + list(X_train.columns)
for i, (name, p_val) in enumerate(zip(feature_names, p_values)):
    print(f"  {name}: p = {p_val:.6f}")

print(f"\nR²         : {model.rsquared_adj:.4f}")

#                 (p < 0.05)
significant_indices = [i for i, p_val in enumerate(p_values) if p_val
    < 0.05 and i > 0]
significant_vars = [X_train.columns[i - 1] for i in
    significant_indices]

print(f"\    n      (p < 0.05): {significant_vars}")

# ==========================================
#
# ==========================================
if significant_vars:
    X_train_sig = X_train[significant_vars]
    X_test_sig = X_test[significant_vars]

    regressor_sig = LinearRegression()
    regressor_sig.fit(X_train_sig, y_train)

    y_pred_train_sig = regressor_sig.predict(X_train_sig)
    y_pred_test_sig = regressor_sig.predict(X_test_sig)

    train_mse_sig = mean_squared_error(y_train, y_pred_train_sig)
```

```
    test_mse_sig = mean_squared_error(y_test, y_pred_test_sig)
    train_r2_sig = r2_score(y_train, y_pred_train_sig)
    test_r2_sig = r2_score(y_test, y_pred_test_sig)

    print("\n" + "=" * 50)
    print("                    ")
    print("=" * 50)
    print(f"MSE            : {train_mse_sig:.4f}")
    print(f"MSE            : {test_mse_sig:.4f}")
    print(f"R²             : {train_r2_sig:.4f}")
    print(f"R²             : {test_r2_sig:.4f}")
else:
    print("\ n          ,            ")
    regressor_sig = regressor_all
    X_test_sig = X_test
    y_pred_test_sig = y_pred_test_all
    test_r2_sig = test_r2_all

# ============================================
#
# ============================================
print("\n" + "=" * 50)
print("                (    )")
print("=" * 50)

feature_importance = pd.DataFrame({
    '    ': X_train.columns,
    '      ': regressor_all.coef_
})
feature_importance = feature_importance.reindex(
    feature_importance['      '].abs().sort_values(ascending=False).index
)
for _, row in feature_importance.iterrows():
    print(f"  {row    ['']}: {row      ['']:.4f}")

# ============================================
#
# ============================================
print("\n" + "=" * 50)
print("           ")
print("=" * 50)

numeric_cols = df.select_dtypes(include=[np.number]).columns
```

```
correlations = df[numeric_cols].corr()['Rings'].sort_values(ascending=False)
for feat, corr in correlations.items():
    print(f"  {feat}: {corr:.4f}")

#
plt.figure(figsize=(10, 8))
correlation_matrix = df[numeric_cols].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, fmt='.2f')
plt.title('                    ')
plt.tight_layout()
plt.savefig('lab7_correlation_matrix.png', dpi=150)
print("\   n                    'lab7_correlation_matrix.png'")

# ==========================================
#
# ==========================================
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

axes[0, 0].scatter(y_train, y_pred_train_all, alpha=0.5, c='blue', edgecolors='k')
axes[0, 0].plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--', lw=2)
axes[0, 0].set_xlabel('          (Rings)')
axes[0, 0].set_ylabel('            (Rings)')
axes[0, 0].set_title(f'           ()\nR² = {train_r2_all:.4f}')
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].scatter(y_test, y_pred_test_all, alpha=0.5, c='green', edgecolors='k')
axes[0, 1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[0, 1].set_xlabel('          (Rings)')
axes[0, 1].set_ylabel('            (Rings)')
axes[0, 1].set_title(f'          ()\nR² = {test_r2_all:.4f}')
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].scatter(y_train, y_pred_train_sig, alpha=0.5, c='blue', edgecolors='k')
axes[1, 0].plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--', lw=2)
axes[1, 0].set_xlabel('          (Rings)')
axes[1, 0].set_ylabel('            (Rings)')
```

```
axes[1, 0].set_title(f'              ()\nR² = {train_r2_sig:.4f}')
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].scatter(y_test, y_pred_test_sig, alpha=0.5, c='green',
    edgecolors='k')
axes[1, 1].plot([y_test.min(), y_test.max()], [y_test.min(),
    y_test.max()], 'r--', lw=2)
axes[1, 1].set_xlabel('          (Rings)')
axes[1, 1].set_ylabel('            (Rings)')
axes[1, 1].set_title(f'            ()\nR² = {test_r2_sig:.4f}')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('lab7_multivariate_results.png', dpi=150)
print("                  'lab7_multivariate_results.png'")

print("                 ")

new_abalone = pd.DataFrame({
    'Length': [0.55],
    'Diameter': [0.44],
    'Height': [0.15],
    'Whole weight': [0.85],
    'Shucked weight': [0.35],
    'Viscera weight': [0.18],
    'Shell weight': [0.28],
    'Sex_I': [0],
    'Sex_M': [1]
})

print("           (        ):")
for col in new_abalone.columns:
    print(f"  {col}: {new_abalone[col].values[0]}")

prediction_all = regressor_all.predict(new_abalone)
print(f"\      n       All-in: {prediction_all[0]:.2f}    ")
print(f"          : {int(round(prediction_all[0]))}")

if significant_vars:
    prediction_sig =
        regressor_sig.predict(new_abalone[significant_vars])
    print(f"                        : {prediction_sig[0]:.2f}    ")

print("                 ")
```

```python

X_simple = df[['Length']]
X_train_simple, X_test_simple, y_train_simple, y_test_simple = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)

regressor_simple = LinearRegression()
regressor_simple.fit(X_train_simple, y_train_simple)
y_pred_test_simple = regressor_simple.predict(X_test_simple)
test_r2_simple = r2_score(y_test_simple, y_pred_test_simple)

print(f"          (Length): R² = {test_r2_simple:.4f}")
print(f"           (All-in): R² = {test_r2_all:.4f}")
print(f"         : {(test_r2_all - test_r2_simple) * 100:.2f}%")

plt.show()
```

Листинг 1.11 — Полный код лабораторной работы №7

Вывод

В ходе выполнения лабораторной работы №7 был разработан пайплайн для решения задачи многомерной регрессии. На основе набора данных Abalone были построены и проанализированы модели линейной регрессии:

- Модель All-in (со всеми признаками) показала $R^2 = 0.5155$, что значительно лучше одномерной модели ($R^2 = 0.3493$).
- Анализ p-значений подтвердил значимость всех признаков ($p < 0.05$).
- Метод Backward Elimination не удалил ни одного признака, так как все признаки оказались значимыми.
- Наибольшее влияние на предсказание оказывает вес раковины (Shell weight) с коэффициентом 8.0895.

Разработанная модель может быть использована для предсказания возраста моллюсков по их физическим характеристикам. Все поставленные задачи выполнены.